

Manipulación del DOM

Teoría:

1. ¿Qué es el DOM?

- Representación estructurada de una página HTML en forma de árbol.
- Permite acceso y modificación de los elementos HTML mediante JavaScript.

2. Selección de elementos:

- `document.getElementById(id)`
- `document.querySelector(selector)`
- `document.querySelectorAll(selector)`

¿CUÁNDO USAR CADA UNO?

`document.getElementById(id)`

-  **Uso ideal:** Cuando sabés que hay un **único elemento** con ese ID.
-  Rápido, directo y eficiente.
-  Ejemplo: seleccionar un botón o título específico.

```
const boton = document.getElementById("miBoton");
```

-  **No usar si el ID no es único** (puede dar resultados inesperados).
-  No sirve para clases, etiquetas o selectores más complejos.

`document.querySelector(selector)`

-  **Uso ideal:** Cuando necesitás seleccionar **el primer elemento que coincida con un selector CSS**.
-  Muy flexible (clases, IDs, atributos, combinaciones).
-  Ejemplo:

```
const primerInput = document.querySelector("input[type='text']");  
  
const titulo = document.querySelector("#miTitulo");  
  
const activo = document.querySelector(".activo");
```

- **!** Sólo devuelve el **primer** elemento que coincida.
- **!** Más lento que `getElementById`, pero mucho más versátil.

`document.querySelectorAll(selector)`

- **💡 Uso ideal:** Cuando querés **una lista de todos los elementos** que coincidan con un selector.
- **✅** Devuelve una **NodeList** (como un array), podés usar `forEach()`.
- **📌** Ejemplo:

```
const items = document.querySelectorAll(".item");  
  
items.forEach(el => el.classList.add("resaltado"));
```

- **!** No devuelve un array real (aunque se puede convertir).
- **!** Puede ser innecesario si sólo necesitás uno.

❌ ¿CUÁNDO NO USARLOS?

Método	No usar cuando...
<code>getElementById()</code>	...el ID no es único, o necesitás varios elementos.
<code>querySelector()</code>	...necesitás más de un elemento (usa <code>querySelectorAll()</code> mejor).
<code>querySelectorAll()</code>	...sólo necesitás el primero o un elemento puntual (usa <code>querySelector()</code>).

3. Modificación de contenido:

- Cambiar texto: `element.textContent = "Nuevo texto";`
- Cambiar HTML interno: `element.innerHTML = "<b>Texto en negrita</b>";`

✅ `element.textContent = "Nuevo texto";`

-  Cambia **solo el texto** del elemento.
-  **Seguro:** no interpreta etiquetas HTML.
-  Ideal para insertar contenido **que no debe tener formato** o evitar inyecciones HTML.

```
<p id="parrafo"></p>
```

```
<script>
```

```
    document.getElementById("parrafo").textContent = "Hola mundo";
```

```
</script>
```

 Resultado visible: Hola mundo (tal cual)

✅ `element.innerHTML = "<b>Texto en negrita</b>";`

-  Cambia el **contenido HTML interno**.
-  Interpreta etiquetas HTML: `<b>`, `<i>`, `<a>`, etc.
-  Usar con cuidado si el contenido viene de usuarios (riesgo de XSS) . El riesgo de un ataque [Cross-Site Scripting \(XSS\)](#) es que un atacante puede inyectar código malicioso en una aplicación web, que se ejecuta en el navegador del usuario final, permitiendo el robo de información sensible como credenciales y cookies de sesión, la redirección a sitios peligrosos, la suplantación de identidad y el compromiso de la confianza del usuario.

```
<p id="parrafo"></p>
```

```
<script>
```

```
    document.getElementById("parrafo").innerHTML = "<b>Hola mundo</b>";
```

```
</script>
```

 Resultado visible: **Hola mundo** (en negrita)

 ¿Cuándo usar cada uno?

Caso	¿Qué usar?
Solo texto plano	textContent ✓
Texto con formato HTML	innerHTML ✓
Insertar etiquetas dinámicas	innerHTML ✓
Seguridad contra scripts	textContent ✓

4. Modificación de estilos:

- Cambiar un estilo: `element.style.color = "blue";`
- Agregar o eliminar clases:

javascript

 Copiar código

```
element.classList.add("clase");
element.classList.remove("clase");
element.classList.toggle("clase");
```

5. Actividad práctica:

- Crear un script que cambie el texto de un encabezado y modifique el color de fondo de un párrafo al hacer clic en un botón.

Ejercicios:

1. Selecciona un elemento por su ID y cambia su texto.
2. Cambia el color de fondo de un elemento al hacer clic en él.
3. Selecciona todos los elementos con una clase específica y cámbiales el texto.
4. Agrega una nueva clase a un elemento seleccionado.
5. Elimina una clase existente de un elemento.
6. Cambia el contenido HTML de un div usando `innerHTML`.
7. Muestra en la consola el texto de un párrafo seleccionado.

8. Usa `querySelectorAll` para seleccionar múltiples elementos y recorrerlos con un bucle.
9. Crea un botón que, al hacer clic, oculte un párrafo.
10. Agrega un nuevo elemento HTML a un contenedor existente.

Eventos en JavaScript

Teoría:

1. ¿Qué son los eventos?

- Acciones o sucesos que ocurren en la página web (clic, teclado, envío de formulario, etc.).

2. Manejo de eventos:

- `addEventListener(evento, función)`

- Ejemplo:

```
javascript Copiar código  
  
document.querySelector("button").addEventListener("click", () => {  
    alert("¡Botón clickeado!");  
});
```

3. Eventos comunes:

- `click`: Detecta clics del ratón.
- `input`: Cambios en un campo de texto.
- `submit`: Envío de formularios.

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>15 Eventos en JavaScript</title>  
</head>  
<body>  
  <h1>Ejemplos de 15 Eventos en JavaScript</h1>
```

```
<!-- 1. click -->
```

```
<section>  
  <h2>1. Evento click</h2>  
  <button id="btnClick">Haz clic aquí</button>  
<script>  
  // Cuando se hace clic en el botón, muestra un mensaje  
  document.getElementById('btnClick').addEventListener('click', function() {  
    alert('¡Hiciste clic en el botón!');
```

```
});  
</script>  
</section>
```

<!-- 2. dblclick -->

```
<section>  
  <h2>2. Evento dblclick</h2>  
  <button id="btnDbClick">Doble clic aquí</button>  
  <script>  
    // Cuando se hace doble clic en el botón, muestra un mensaje  
    document.getElementById('btnDbClick').addEventListener('dblclick', function() {  
      alert('¡Doble clic detectado!');  
    });  
  </script>  
</section>
```

<!-- 3. mouseover -->

```
<section>  
  <h2>3. Evento mouseover</h2>  
  <div id="boxHover" style="padding:20px;border:1px solid #000;">Pasa el mouse por  
aquí</div>  
  <script>  
    // Cambia el color del fondo cuando el mouse pasa por encima  
    const boxHover = document.getElementById('boxHover');  
    boxHover.addEventListener('mouseover', function() {  
      boxHover.style.backgroundColor = '#ffd700';  
    });  
    boxHover.addEventListener('mouseout', function() {  
      boxHover.style.backgroundColor = '';  
    });  
  </script>  
</section>
```

<!-- 4. keydown -->

```
<section>  
  <h2>4. Evento keydown</h2>  
  <input type="text" id="inputKeydown" placeholder="Presiona una tecla">  
  <script>  
    // Muestra en consola la tecla presionada  
    document.getElementById('inputKeydown').addEventListener('keydown', function(e)  
{  
      console.log('Tecla presionada:', e.key);  
    });  
  </script>  
</section>
```

<!-- 5. keyup -->

```
<section>  
  <h2>5. Evento keyup</h2>  
  <input type="text" id="inputKeyup" placeholder="Escribe algo y suelta la tecla">  
  <script>  
    // Muestra en consola la tecla soltada  
    document.getElementById('inputKeyup').addEventListener('keyup', function(e) {  
      console.log('Tecla soltada:', e.key);  
    });  
  </script>  
</section>
```

<!-- 6. input -->

```
<section>
  <h2>6. Evento input</h2>
  <input type="text" id="inputRealTime" placeholder="Escribe aquí">
  <p id="outputInput"></p>
  <script>
    // Muestra en tiempo real lo que se escribe
    document.getElementById('inputRealTime').addEventListener('input', function() {
      document.getElementById('outputInput').textContent = `Escribiste: ${this.value}`;
    });
  </script>
</section>
```

<!-- 7. change -->

```
<section>
  <h2>7. Evento change</h2>
  <select id="selectChange">
    <option value="">Selecciona una opción</option>
    <option value="Opción 1">Opción 1</option>
    <option value="Opción 2">Opción 2</option>
  </select>
  <p id="outputSelect"></p>
  <script>
    // Muestra la opción seleccionada al cambiar el select
    document.getElementById('selectChange').addEventListener('change', function() {
      document.getElementById('outputSelect').textContent = `Seleccionaste:
${this.value}`;
    });
  </script>
</section>
```

<!-- 8. submit -->

```
<section>
  <h2>8. Evento submit</h2>
  <form id="formSubmit">
    <input type="text" placeholder="Tu nombre" required>
    <button type="submit">Enviar</button>
  </form>
  <script>
    // Evita que el formulario recargue la página y muestra un mensaje
    document.getElementById('formSubmit').addEventListener('submit', function(e) {
      e.preventDefault();
      alert('Formulario enviado correctamente');
    });
  </script>
</section>
```

<!-- 9. load -->

```
<section>
  <h2>9. Evento load</h2>
  <p id="msgLoad">Cargando...</p>
  <script>
    // Cuando la página carga completamente, cambia el texto
    window.addEventListener('load', function() {
      document.getElementById('msgLoad').textContent = 'Página cargada
completamente';
    });
  </script>
</section>
```

```
});  
</script>  
</section>
```

<!-- 10. scroll -->

```
<section>  
  <h2>10. Evento scroll</h2>  
  <div id="scrollBox" style="height:100px; overflow-y:scroll; border:1px solid #000;  
padding:5px;">  
    <p>Contenido 1</p>  
    <p>Contenido 2</p>  
    <p>Contenido 3</p>  
    <p>Contenido 4</p>  
    <p>Contenido 5</p>  
  </div>  
  <p id="scrollPos">Posición del scroll: 0px</p>  
  <script>  
    // Detecta el scroll dentro del div y muestra la posición  
    const scrollBox = document.getElementById('scrollBox');  
    const scrollPos = document.getElementById('scrollPos');  
    scrollBox.addEventListener('scroll', function() {  
      scrollPos.textContent = `Posición del scroll: ${scrollBox.scrollTop}px`;  
    });  
  </script>  
</section>
```

<!-- 11. resize -->

```
<section>  
  <h2>11. Evento resize</h2>  
  <p id="sizeOutput"></p>  
  <script>  
    const sizeOutput = document.getElementById('sizeOutput');  
    // Inicializa al cargar  
    sizeOutput.textContent = `Ancho: ${window.innerWidth}px, Alto:  
${window.innerHeight}px`;  
    // Actualiza al redimensionar la ventana  
    window.addEventListener('resize', function() {  
      sizeOutput.textContent = `Ancho: ${window.innerWidth}px, Alto:  
${window.innerHeight}px`;  
    });  
  </script>  
</section>
```

<!-- 12. contextmenu -->

```
<section>  
  <h2>12. Evento contextmenu</h2>  
  <div id="rightClickBox" style="padding:20px;border:1px solid #000;">Haz clic derecho  
aquí</div>  
  <script>  
    // Evita menú contextual y muestra alerta  
    document.getElementById('rightClickBox').addEventListener('contextmenu',  
function(e) {  
      e.preventDefault();  
      alert('Clic derecho detectado');  
    });  
  </script>  
</section>
```

<!-- 13. focus -->

```
<section>
  <h2>13. Evento focus</h2>
  <input type="text" id="inputFocus" placeholder="Haz clic aquí">
  <script>
    // Cambia fondo al recibir foco
    document.getElementById('inputFocus').addEventListener('focus', function() {
      this.style.backgroundColor = '#d0f0ff';
    });
  </script>
</section>
```

<!-- 14. blur -->

```
<section>
  <h2>14. Evento blur</h2>
  <input type="text" id="inputBlur" placeholder="Haz clic y luego fuera">
  <script>
    // Cambia fondo al perder foco
    const inputBlur = document.getElementById('inputBlur');
    inputBlur.addEventListener('blur', function() {
      this.style.backgroundColor = "";
    });
  </script>
</section>
```

<!-- 15. drag & drop -->

```
<section>
  <h2>15. Evento drag & drop</h2>
  <div id="dragBox" draggable="true" style="padding:20px;border:1px solid #000;
display:inline-block;">Arrástrame</div>
  <div id="dropBox" style="width:200px;height:100px;border:1px solid #000;
display:inline-block; vertical-align:top; margin-left:10px;">Suéltame aquí</div>
  <script>
    const dragBox = document.getElementById('dragBox');
    const dropBox = document.getElementById('dropBox');

    dragBox.addEventListener('dragstart', function(e) {
      e.dataTransfer.setData('text', e.target.id);
    });

    dropBox.addEventListener('dragover', function(e) {
      e.preventDefault();
    });

    dropBox.addEventListener('drop', function(e) {
      e.preventDefault();
      const data = e.dataTransfer.getData('text');
      const draggedElement = document.getElementById(data);
      dropBox.appendChild(draggedElement);
      draggedElement.style.backgroundColor = '#aaffaa';
    });
  </script>
</section>
</body>
</html>
```

4. Actividad práctica:

- Crear un formulario interactivo que:
 - Valide un campo de texto en tiempo real.
 - Muestre un mensaje de confirmación al enviarlo.

Ejercicios:

1. Crea un botón que muestre un mensaje en consola al hacer clic.
 2. Haz que un campo de texto cambie su color de fondo al escribir en él.
 3. Implementa un contador que aumente cada vez que se haga clic en un botón.
 4. Muestra un mensaje de error si un campo de texto está vacío al enviar un formulario.
 5. Cambia la posición de un elemento cuando se pase el ratón sobre él.
 6. Detecta y muestra en la consola la tecla presionada por el usuario.
 7. Implementa un botón que oculte o muestre un elemento al hacer clic (toggle).
 8. Crea un campo de selección (dropdown) que actualice un párrafo con la opción seleccionada.
 9. Implementa un formulario con validación para correos electrónicos.
 10. Haz un evento que cambie el texto de un botón al hacer clic en él.
-

Arreglos y Objetos

Teoría:

1. Arreglos:

- Estructuras para almacenar múltiples valores.
- Métodos comunes:
 - `push()`, `pop()`, `shift()`, `unshift()`.
 - `forEach()`, `map()`, `filter()`, `reduce()`.

2. Objetos:

- Almacenan datos como pares clave-valor.
- Ejemplo:

```
javascript Copiar código  
  
const persona = {  
  nombre: "Juan",  
  edad: 25,  
  saludar: function() {  
    return "Hola, " + this.nombre;  
  }  
};  
console.log(persona.saludar());
```

3. Actividad práctica:

- Crear un arreglo de objetos donde cada objeto represente un producto con propiedades como **nombre**, **precio**, y **cantidad**. Implementar funciones para:
 - Agregar productos.
 - Calcular el costo total.
 - Filtrar productos por precio.

Ejercicios:

1. Crea un arreglo con 5 números y muestra el tercero en consola.
2. Agrega un nuevo elemento a un arreglo usando **push**.
3. Elimina el último elemento de un arreglo y muéstralo en consola.
4. Usa **map** para multiplicar por 2 cada número de un arreglo.
5. Filtra un arreglo para obtener solo los números mayores a 10.
6. Declara un objeto con las propiedades **nombre**, **edad**, y **profesión**.
7. Accede y muestra el valor de una propiedad de un objeto.
8. Agrega un método a un objeto que devuelva un saludo personalizado.
9. Usa **reduce** para sumar todos los números de un arreglo.
10. Combina arreglos y objetos: Crea un arreglo de objetos y recórrelo para mostrar el nombre de cada elemento.

Actividad principal

Crear un arreglo de objetos donde cada objeto representa un producto con propiedades como nombre, precio y cantidad. Implementar funciones para:

Código:

// Arreglo de productos

```
let productos = [  
  { nombre: "Laptop", precio: 800000, cantidad: 2 },  
  { nombre: "Mouse", precio: 5000, cantidad: 5 },  
  { nombre: "Teclado", precio: 12000, cantidad: 3 }  
];
```

// Función para agregar productos

```
function agregarProducto(nombre, precio, cantidad) {  
  productos.push({ nombre, precio, cantidad });  
}
```

// Función para calcular el costo total

```
function calcularCostoTotal() {  
  return productos.reduce((total, producto) => total + (producto.precio * producto.cantidad),  
    0);  
}
```

// Función para filtrar productos por precio

```
function filtrarPorPrecio(precioMinimo) {  
  return productos.filter(producto => producto.precio >= precioMinimo);  
}
```

// Ejemplo de uso:

```
agregarProducto("Monitor", 70000, 1);  
console.log("Productos:", productos);  
console.log("Costo total:", calcularCostoTotal());  
console.log("Productos con precio mayor o igual a 10000:", filtrarPorPrecio(10000));
```



Ejercicios individuales

1. Crea un arreglo con 5 números y muestra el tercero en consola.

```
let numeros = [4, 8, 15, 16, 23];  
console.log(numeros[2]); // 15
```

2. Agrega un nuevo elemento a un arreglo usando **push.**

```
numeros.push(42);  
console.log(numeros);
```

3. Elimina el último elemento de un arreglo y muéstralo en consola.

```
let ultimo = numeros.pop();  
console.log("Elemento eliminado:", ultimo);  
console.log("Arreglo actualizado:", numeros);
```

4. Usa **map para multiplicar por 2 cada número de un arreglo.**

```
let duplicados = numeros.map(num => num * 2);  
console.log(duplicados);
```

5. Filtra un arreglo para obtener solo los números mayores a 10.

```
let mayoresA10 = numeros.filter(num => num > 10);  
console.log(mayoresA10);
```

6. Declara un objeto con las propiedades nombre, edad y profesión.

```
let persona = { nombre: "Ana", edad: 30, profesion: "Programadora" };  
};
```

7. Accede y muestra el valor de una propiedad de un objeto.

```
console.log(persona.nombre);
```

8. Agrega un método a un objeto que devuelva un saludo personalizado.

```
persona.saludar = function() {
```

```
    return `Hola, soy ${this.nombre} y soy ${this.profesion}`;  
  };  
  console.log(persona.saludar());
```

9. Usa **reduce** para sumar todos los números de un arreglo.

```
let suma = numeros.reduce((acum, num) => acum + num, 0);  
console.log("Suma total:", suma);
```

10. Combina arreglos y objetos: crea un arreglo de objetos y recórrelo para mostrar el nombre de cada elemento.

```
let frutas = [  
  { nombre: "Manzana", color: "Rojo" },  
  { nombre: "Banana", color: "Amarillo" },  
  { nombre: "Uva", color: "Morado" }  
];  
  
frutas.forEach(fruta => console.log(fruta.nombre));
```